

Session 2

Linear models

Hector J. Hortua, PhD

Basado presentación:

Dr Ivan Olier

Learning outcomes

By the end of this session we should be able to:

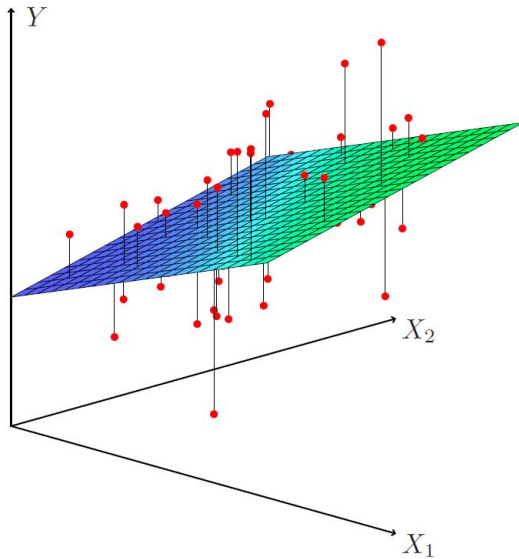
- Understand the ML modelling process.
- Describe the general formulas of linear regression and logistic regression.
- Estimate the performance of a ML model using a test set.
- Write small Python functions that implements linear and logistic regressions.
- Use Scikit-Learn to implement ML models based on linear and logistic regression algorithms.

Linear methods

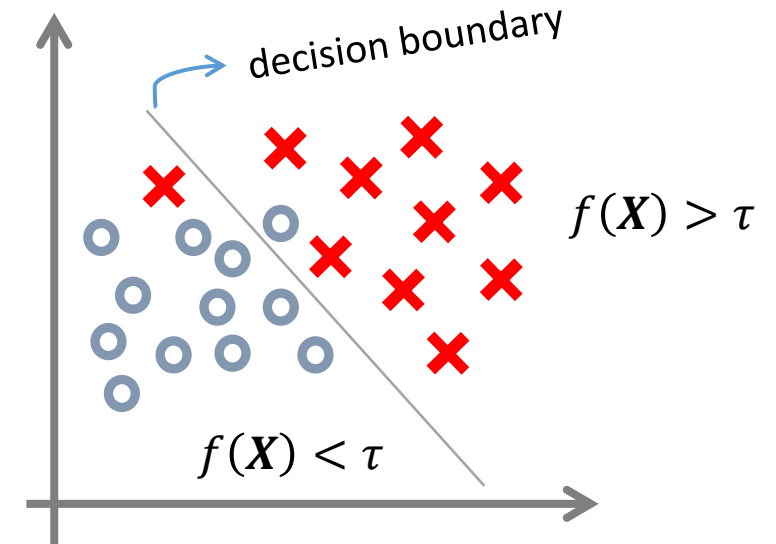
- Linear methods assume input variables are linearly independent.
- Their outputs are linear combinations of the inputs

$$f(\mathbf{X}) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots \beta_p X_p$$

Regression: $Y = f(\mathbf{X})$ is a hyperplane



Classification: decision boundary is the hyperplane $f(\mathbf{X})$.



Multiple (Multivariate) linear regression model

- Let's assume X^T a vector of inputs $(X_1, \dots, X_p)$. A multiple linear regression model is defined as

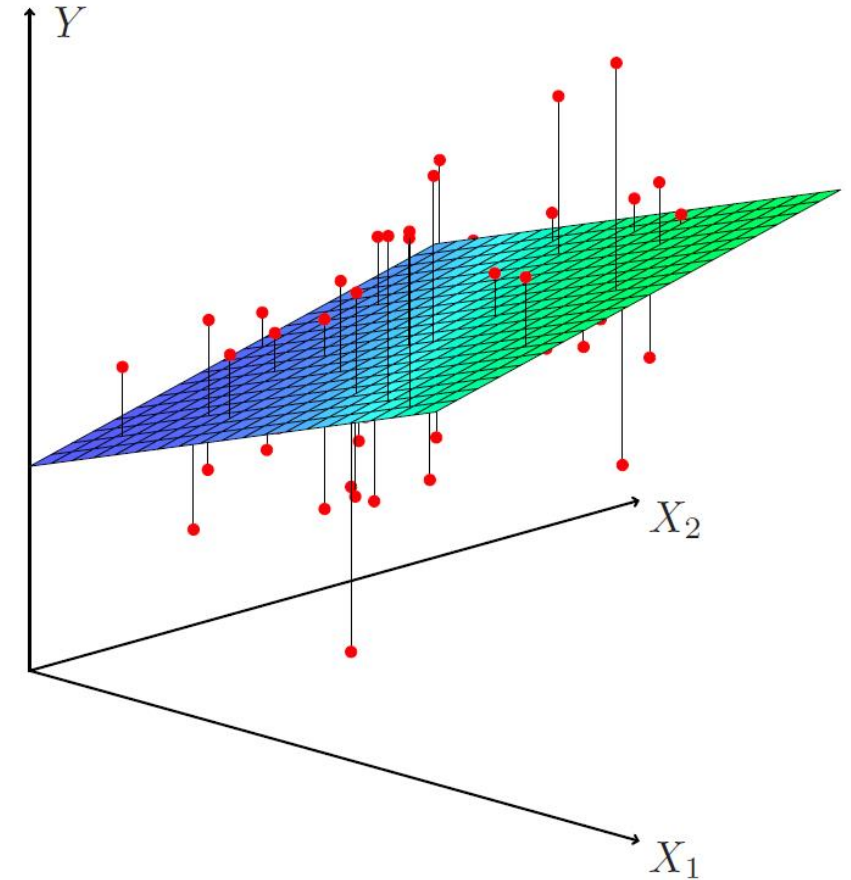
$$Y = \beta_0 + \sum_{j=1}^P X_j \beta_j + \epsilon$$

- Or, if 1 is included in X ,

$$Y = \sum_{j=0}^P X_j \beta_j + \epsilon = X^T \beta + \epsilon$$

- Predicting a **new** output variable value, assuming $\hat{\beta}$ is already estimated, is given by

$$\hat{Y} = X^T \hat{\beta}$$



Multiple linear regression – Estimation of parameters

- How do we **fit** the multiple linear model to a set of training data? least squares is an option.
- It estimates $\hat{\beta}$ by **minimising** the residual sum of squares

$$RSS(\hat{\beta}) = \sum_{i=1}^n \epsilon_i^2 = \sum_{i=1}^n (y_i - x_i^T \hat{\beta})^2 = (\mathbf{y} - \mathbf{X}\hat{\beta})^T (\mathbf{y} - \mathbf{X}\hat{\beta})$$

- Where $(\mathbf{X}, \mathbf{y})$ is the training dataset, $\mathbf{X}$, an $n \times p$ matrix, $\mathbf{y}$, an n -vector of outputs.
- ... and $\hat{\beta} = (\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p)^T$ a vector of model parameters.
- Since RSS is a quadratic function of the parameters, there is the **guarantee** of at least a local minimum, although it may not be unique. So, **differentiating** w.r.t. β , **equating** to 0 and **solving** β , we get

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Example: Advertising data

- Consider the following dataset[*], which was collected to predict impact of advertising on sales:

TV	radio	newspaper	sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	9.3
151.5	41.3	58.5	18.5
180.8	10.8	58.4	12.9
...	...	...	...

- We should expect a model that predicts *sales* based on *TV*, *radio*, and *newspaper* predictor variables:

$$\text{sales} = f(\text{TV}, \text{radio}, \text{newspaper})$$



[*] <http://www-bcf.usc.edu/~gareth/ISL/data.html>

Example – Advertising data

- Suggested multiple linear regression model:

$$sales = \beta_0 + \beta_1 \cdot TV + \beta_2 \cdot radio + \beta_3 \cdot newspaper + \epsilon$$

- Coefficients are estimated after model training (or fitting). This is, by applying least squares formula:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- Then:

$$\beta_0 = 2.939$$

$$\beta_2 = 0.189$$

$$\beta_1 = 0.046$$

$$\beta_3 = -0.001$$

Model performance – regression task

Residual sum of squares (RSS)

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

R-squared (R^2) – fraction of variance in the data
explained. Total Sum of Squares(TSS)

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{RSS}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

Mean squared error (MSE) – average amount that the response will **deviate** from the true.

$$MSE = \frac{1}{n} RSS$$

Root mean squared error (RMSE) – average amount that the response will **deviate** from the true.

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} RSS}$$

Interpreting regression coefficients

- A multiple regression model

$$Y = \beta_0 + \sum_{j=1}^P X_j \beta_j + \epsilon$$

- We interpret β_j as the average effect on Y of a **one** unit increase in X_j , **holding** all other predictors fixed.

<https://scikit-learn.org/stable/index.html>

<https://feature-engine.trainindata.com/en/latest/>

Building a Linear Regression model in 4 steps

1. Scikit-learn's Linear Regression module:

```
from sklearn.linear_model import LinearRegression
```

2. Sets up the learner (or algorithm):

```
lin_md1 = LinearRegression()
```

3. Trains the model using a training set:

```
lin_md1.fit(X_train, y_train)
```

4. Predicts new targets:

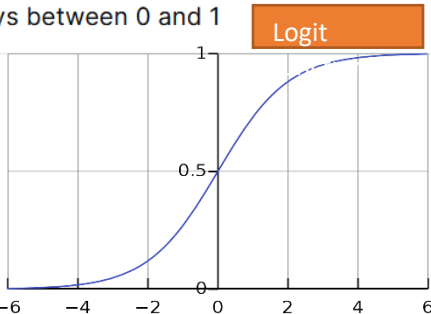
```
lin_md1.predict(X_test)
```

Issues

- The ideal scenario is when the predictors are **uncorrelated** – a balanced design:
- Each coefficient can be estimated and tested **separately**.
- Interpretations such as “a unit change in X_j is associated with a β_j change in Y , while all the other variables stay fixed”, are possible.
- Correlations amongst predictors cause problems:
 - The variance of all coefficients **tends** to increase, sometimes dramatically.
 - Interpretations become hazardous – when X_j changes, **everything** else changes.

Linear models for regression

Component	Explanation
Odds = $\frac{p}{1-p}$	The ratio of probability of event to probability of non-event. Odds range from 0 to ∞
Log-odds (logit) = $\log(\frac{p}{1-p})$	Transforms odds to range from $-\infty$ to $+\infty$ — unbounded, suitable for a linear model
Linear combination	The right-hand side is exactly the same as linear regression — a weighted sum of predictors
Inverse transformation	Solving for $p(X)$ gives the logistic function (S-curve) that always stays between 0 and 1



As linear predictor $\rightarrow +\infty$, $p(X) \rightarrow 1$
As linear predictor $\rightarrow -\infty$, $p(X) \rightarrow 0$
When linear predictor = 0, $p(X) = 0.5$

Aspect	Linear Regression	Logistic Regression
Outcome	Continuous (real number)	Binary (0/1)
Model	$Y = X\beta + \epsilon$	$\log(\frac{p}{1-p}) = X\beta$
Estimation	Least squares	Maximum likelihood
Coefficient interpretation	Change in Y per unit change in X	Change in log-odds per unit change in X ; e^β = odds ratio
Decision boundary	N/A	Linear in X -space

The **Odds Ratio** is the ratio of two odds:

$$OR = \frac{\text{Odds in Group A}}{\text{Odds in Group B}}$$

It tells you how many times higher (or lower) the odds are in one group compared to another.

$$\log \left[\frac{p(X)}{1 - p(X)} \right] = \beta_0 + \beta_1 X_1$$

Group	Log-Odds	Odds
X = 0 (reference)	β_0	e^{β_0}
X = 1 (exposed)	$\beta_0 + \beta_1$	$e^{\beta_0 + \beta_1} = e^{\beta_0} \cdot e^{\beta_1}$

Odds Ratio comparing X=1 to X=0:

$$OR = \frac{e^{\beta_0 + \beta_1} \cdot e^{\beta_1}}{e^{\beta_0}} = e^{\beta_1}$$

β	e^β (OR)	Interpretation
0.69	2.0	Odds are 2 times higher (100% increase in odds)
0.41	1.5	Odds are 1.5 times higher (50% increase)
0	1.0	No effect
-0.41	0.67	Odds are 0.67 times (33% decrease)
-0.69	0.5	Odds are half (50% decrease)

$CI = e^{\beta_1 \pm 1.96 \times SE(\beta_1)}$ 2.34, 95% CI = [1.45, 3.78] — significant because interval doesn't include 1.

Linear models for regression

Component	Explanation
Odds = $\frac{p}{1-p}$	The ratio of probability of event to probability of non-event. Odds range from 0 to ∞

Log-odds (logit)	Transforms odds to range from $-\infty$ to $+\infty$ — unbounded.
-------------------------	---

Suppose you model whether a patient has heart disease:

Linear combination
Inverse transform

$$\log\left(\frac{p}{1-p}\right) = -3.5 + 0.07 \cdot \text{Age} + 1.2 \cdot \text{SexMale}$$

As linear predictor →
As linear predictor →
When linear predictor

- **Age:** $e^{0.07} \approx 1.072$ — each year increases odds by 7.2%
- **SexMale:** $e^{1.2} \approx 3.32$ — males have 3.32× higher odds than females (reference)
- **Intercept:** When Age=0 and Sex not male, log-odds = -3.5 → probability = $e^{-3.5} / (1 + e^{-3.5}) \approx 0.029$ (baseline risk)

Aspect		
Outcome		
Model	$Y = X\beta + \epsilon$	$\log(\frac{p}{1-p}) = X\beta$
Estimation	Least squares	Maximum likelihood
Coefficient interpretation	Change in Y per unit change in X	Change in log-odds per unit change in X ; e^β = odds ratio
Decision boundary	N/A	Linear in X -space

The **Odds Ratio** is the ratio of two odds:

$$OR = \frac{\text{Odds in Group A}}{\text{Odds in Group B}}$$

It tells you how many times higher (or lower) the odds are in one group compared to another.

Odds

$$e^{\beta_0 + \beta_1} = e^{\beta_0} \cdot e^{\beta_1}$$

0.69	2.0	Odds are 2 times higher (100% increase in odds)
0.41	1.5	Odds are 1.5 times higher (50% increase)
0	1.0	No effect
-0.41	0.67	Odds are 0.67 times (33% decrease)
-0.69	0.5	Odds are half (50% decrease)

$CI = e^{\beta_1 \pm 1.96 \times SE(\beta_1)}$ 2.34, 95% CI = [1.45, 3.78] — significant because interval doesn't include 1.

(Multiple) logistic regression

- LR is a statistical method that model the risk of an outcome as a function of the linear combination of a set of input variables:

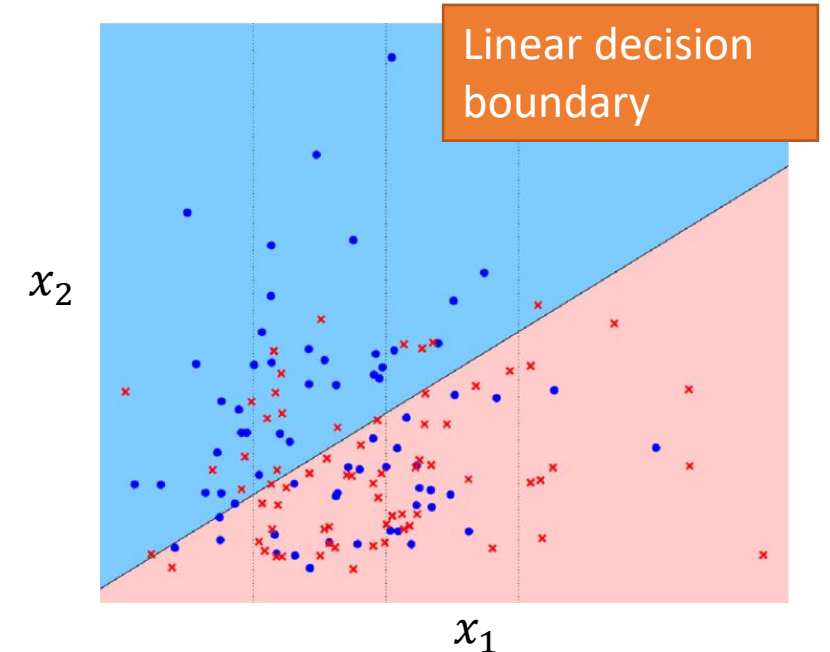
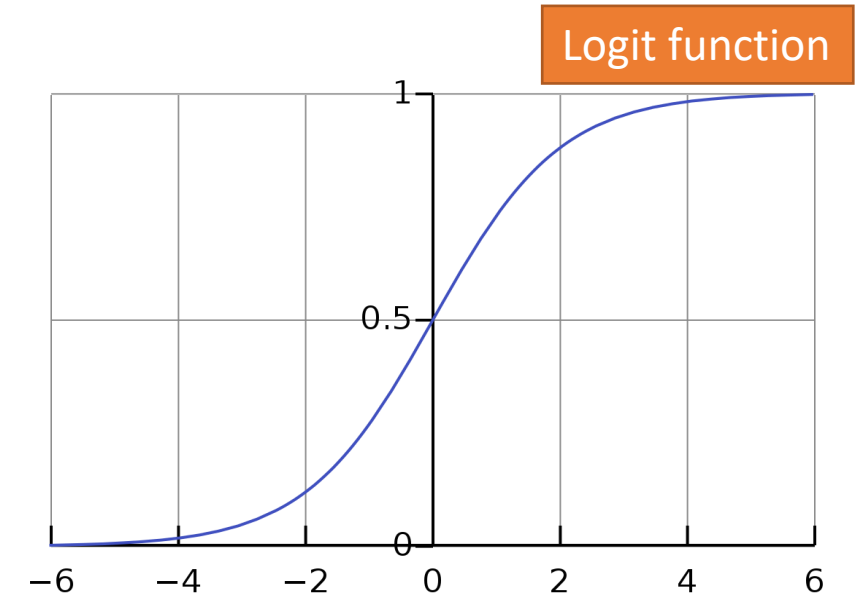
$$\log \left[\frac{p(X)}{1 - p(X)} \right] = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$$

$$p(X) = \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}$$

- Model parameters ($\boldsymbol{\beta} = \{\beta_0, \beta_1, \dots, \beta_p\}$) are estimated via Maximum Likelihood (the probability of the data given the model, $\mathcal{L}(\boldsymbol{\beta})$).

$$\mathcal{L}(\boldsymbol{\beta}) = \left[\prod_{i=1}^N p(Y_i | x_i, \boldsymbol{\beta})^{Y_i} \right] \left[\prod_{i=1}^N (1 - p(Y_i | x_i, \boldsymbol{\beta}))^{(1-Y_i)} \right]$$

Where, Y_i is the binary outcome for the i -th observation (0 or 1).



Example – Predicting gestational diabetes

Task – to predict the risk of diabetes in pregnant women. This is a classification task.

Dataset

Target

Instances

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1
5	116	74	0	0	25.6	0.201	30	0
3	78	50	32	88	31	0.248	26	1
10	115	0	0	0	35.3	0.134	29	0
2	197	70	45	543	30.5	0.158	53	1
8	125	96	0	0	0	0.232	54	1
4	110	92	0	0	37.6	0.191	30	0
10	168	74	0	0	38	0.537	34	1
10	139	80	0	0	27.1	1.441	57	0
1	189	60	23	846	30.1	0.398	59	1
5	166	72	19	175	25.8	0.587	51	1
...								

Input variables

To **build** a model using the available data such that it is able to predict the risk of diabetes in **unseen** cases (new women)

Example – Predicting gestational diabetes

Algorithm – We will use *logistic regression*, which is suitable for classification and it is easy to implement.

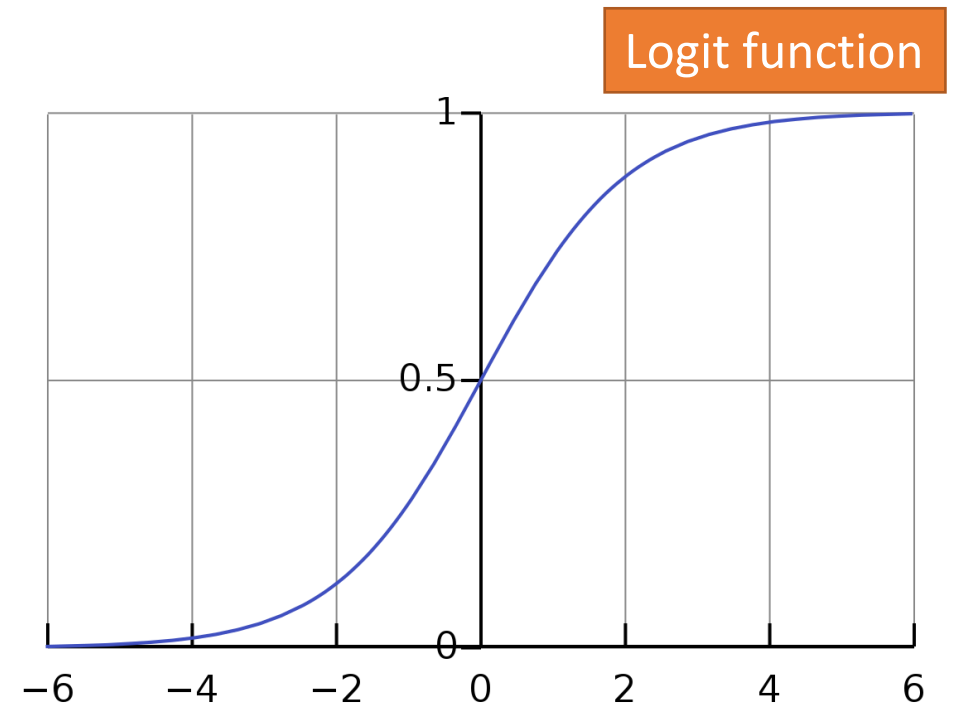
Logistic regression predicts the *class probability* given a set of input variables

$$P(X) = \frac{e^{(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p)}}{1 + e^{(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p)}}$$

Input variables: $X_1, \dots, X_p$

Model parameters: $\beta_0, \dots, \beta_p$

Target: $P(X)$ – probability of suffering from diabetes.



Building a Logistic Regression model in 4 steps

1. Scikit-learn's Logistic Regression module:

```
from sklearn.linear_model import LogisticRegression
```

2. Sets up the learner (or algorithm):

```
mdl = LogisticRegression(max_iter = 1000)
```

3. Trains the model using a training set:

```
mdl.fit(X_train, y_train)
```

4. Predicts new labels (classes, targets):

```
mdl.predict(X_test)
```

Example – Predicting gestational diabetes

Prediction – using test data.

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
2	197	70	45	543	30.5	0.158	53
1	189	60	23	846	30.1	0.398	59
5	166	72	19	175	25.8	0.587	51
1	115	70	30	96	34.6	0.529	32
3	126	88	41	235	39.3	0.704	27
10	125	70	26	115	31.1	0.205	41
1	97	66	15	140	23.2	0.487	22
3	180	64	25	70	34	0.271	26
9	171	110	24	240	45.4	0.721	54
...							

Test data – they have not been used for training

$$P(X) = \frac{e^{(-10.956 + 0.18 \times 2 + 0.03 \times 197 + 0.004 \times 70 - 0.008 \times 45 + 0.001 \times 543 + 0.11 \times 30.5 + 1.70 \times 0.158 - 0.003 \times 53)}}{1 + e^{(-10.956 + 0.18 \times 2 + 0.03 \times 197 + 0.004 \times 70 - 0.008 \times 45 + 0.001 \times 543 + 0.11 \times 30.5 + 1.70 \times 0.158 - 0.003 \times 53)}}$$

$$P(X) = 0.788 \quad \leftarrow \text{High risk of developing diabetes!}$$

Categorical predictors

- A categorical (or qualitative, or factor) predictor takes **categorical** values (i.e. levels with **no** particular order) only.
 - Examples: gender (female, male), marital status (single, married, etc), ethnicity (Caucasian, African American, Asian).
- How to **code** categorical predictors?
 - WRONG WAY – Create a numerical variable, assign an integer value to each level. This wrongly creates an order between the levels.
 - CORRECT WAY – Having m levels, create $(m - 1)$ binary variables, one for each level. Each variable can take 1 to represent a corresponding level. There is one level without variable, it is represented when all the binary variables take 0. It is called baseline.

Example – Coding BMI

- Let's assume we have a categorical variable for BMI with levels:

BMI			
Underweight	Normal	Overweight	Obese

- It must be coded using three new binary variables as follows:

New binary variable	value	meaning
BMI[Normal]	1	BMI is normal
	0	BMI is not normal
BMI[Overweight]	1	BMI is overweight
	0	BMI is not overweight
BMI[Obese]	1	BMI is obese
	0	BMI is not obese

- The baseline, which is not a variable, represents the absence of the coded levels.

Baseline (not a variable)	BMI[Normal] = 0, BMI[Overweight] = 0, & BMI[Obese] = 0	BMI is neither normal, nor overweight, nor obese.
---------------------------	--	---

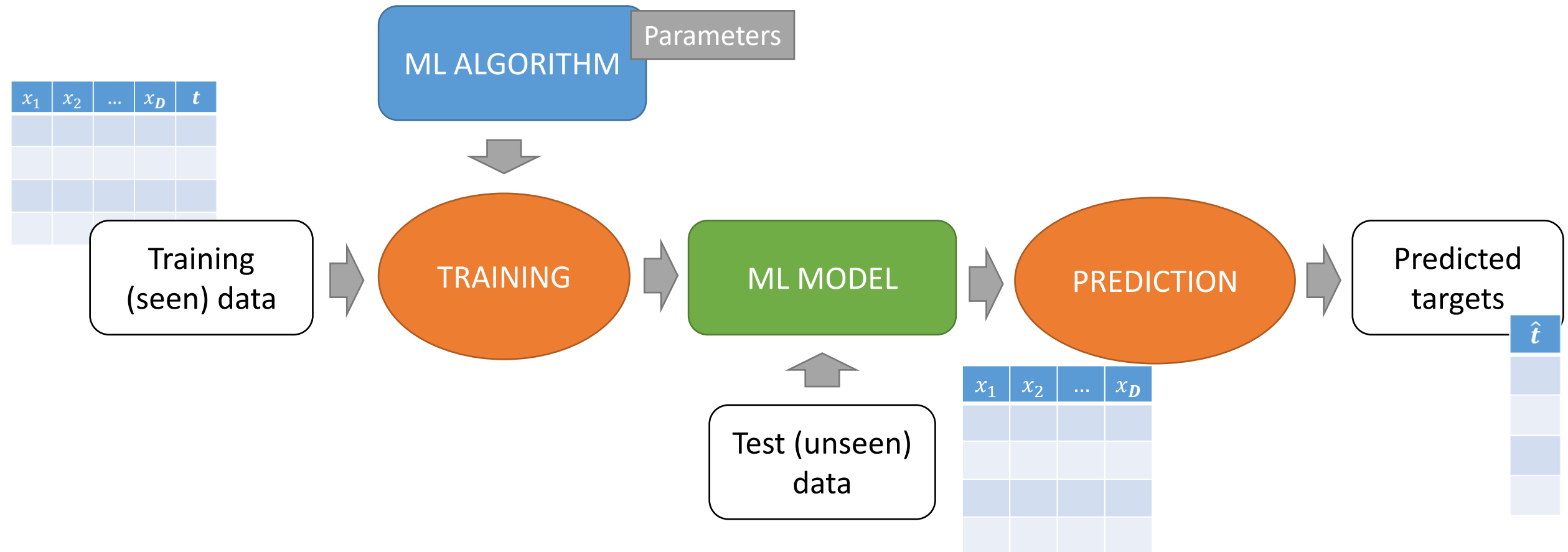
Example – Coding BMI

- Let's assume we have a categorical variable for BMI with levels:

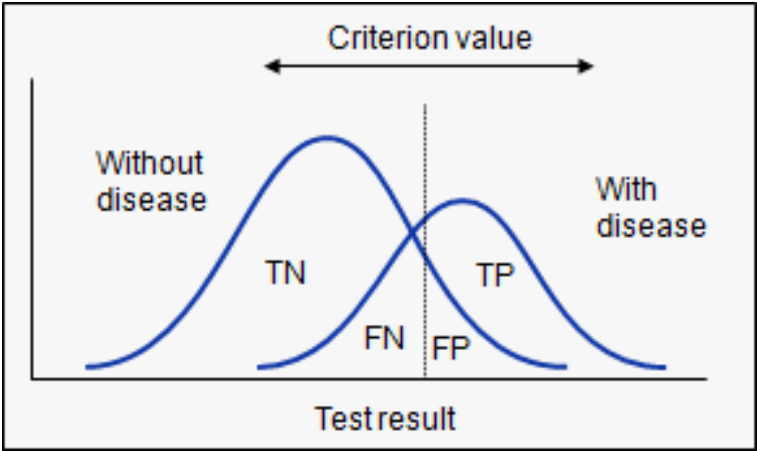
BMI Category	BMI			Interpretation
	BMI[Normal]	BMI[Overweight]	BMI[Obese]	
Underweight	0	0	0	Baseline (all dummies = 0)
Normal	1	0	0	Only Normal dummy = 1
Overweight	0	1	0	Only Overweight dummy = 1
Obese	0	0	1	Only Obese dummy = 1

Model fitting (or training)

- A ML/Statistical model is **fitted** or trained by using data (training data) to estimate its parameters.
- In turns, a fitted model can be used to make **predictions** using **unseen** data (test data).



Model performance – classification task



Inputs (X)	C	$Pr(\hat{C} X)$	$\hat{C}_1$	$\hat{C}_2$	$\hat{C}_3$
...	0	0.1	0	0	0
...	0	0.2	0	0	0
...	0	0.3	0	1	0
...	1	0.4	0	1	0
...	0	0.5	1	1	0
...	1	0.6	1	1	0
...	0	0.7	1	1	0
...	1	0.8	1	1	1
...	1	0.9	1	1	1

Confusion matrix

Actual class	Predicted class	
	TP True Positive	FN False Negative
	FP False Positive	TN True Negative

Classifier 1

Actual class	Predicted class	
	3	1
	2	3

Classifier 2

Actual class	Predicted class	
	4	0
	3	2

Classifier 3

Actual class	Predicted class	
	2	2
	0	5

Model performance – classification task

- **Sensitivity, true positive rate or recall**
measures the ability of a test to correctly identify those with the disease (positive cases)
- **Specificity, true negative rate**
measures the ability of a test to correctly identify those without the disease (negative cases)
- **Precision**
proportion of the predicted cases with the disease (positive cases) that were correct
- **Accuracy**
indicates how correct a classifier is
- **Balanced accuracy**
same as before but takes into account imbalanced classes

True Positive Rate (TPR), False Positive Rate (FPR)

$$TPR, Recall, Sensitivity = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{TN + FP} \quad FPR = \frac{FP}{TN + FP}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Accuracy = \frac{TP + TN}{TP + FN + FP + TN}$$

$$Balanced\ acc = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{FP + TN} \right)$$

Model performance – classification task

- **Sensitivity, true positive rate or recall**

measures the ability of a test to correctly identify those with the disease

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

- **Specificity**, measures the ability of a test to correctly identify those without the disease

$$\text{Specificity} = \frac{TN}{TN + FP}$$

- **Precision** measures the proportion of positive cases that are correctly identified

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Accuracy** indicates the proportion of correct predictions

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Balanced accuracy** same as before but takes into account imbalanced classes

$$\text{Balanced acc} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{FP + TN} \right)$$

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Model performance

- **Accuracy**
indicates how correct a classifier is
- **Balanced accuracy**
same as before but takes into account imbalanced classes

$$Accuracy = \frac{TP + TN}{TP + FN + FP + TN}$$

$$Balanced\ acc = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{FP + TN} \right)$$

Balanced classes:
C1=100, C2=100

		predicted labels	
		C1	C2
true labels	C1	90	10
	C2	20	80

regular accuracy:	balanced accuracy:
$\frac{90 + 80}{90 + 10 + 20 + 80} = 0.85$	$\left(\frac{90}{90 + 10} + \frac{80}{20 + 80} \right) / 2 = 0.85$

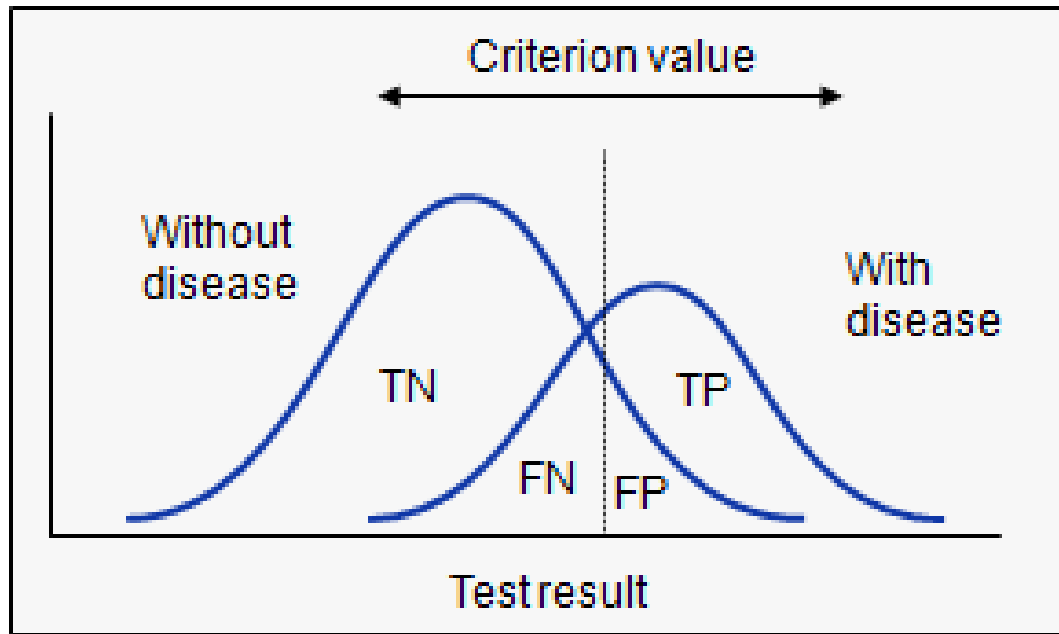
Imbalanced classes:
C1=100, C2=5

		predicted labels	
		C1	C2
true labels	C1	90	10
	C2	4	1

regular accuracy:	balanced accuracy:
$\frac{90 + 1}{90 + 10 + 4 + 1} = 0.867$	$\left(\frac{90}{90 + 10} + \frac{1}{4 + 1} \right) / 2 = 0.55$

Model performance

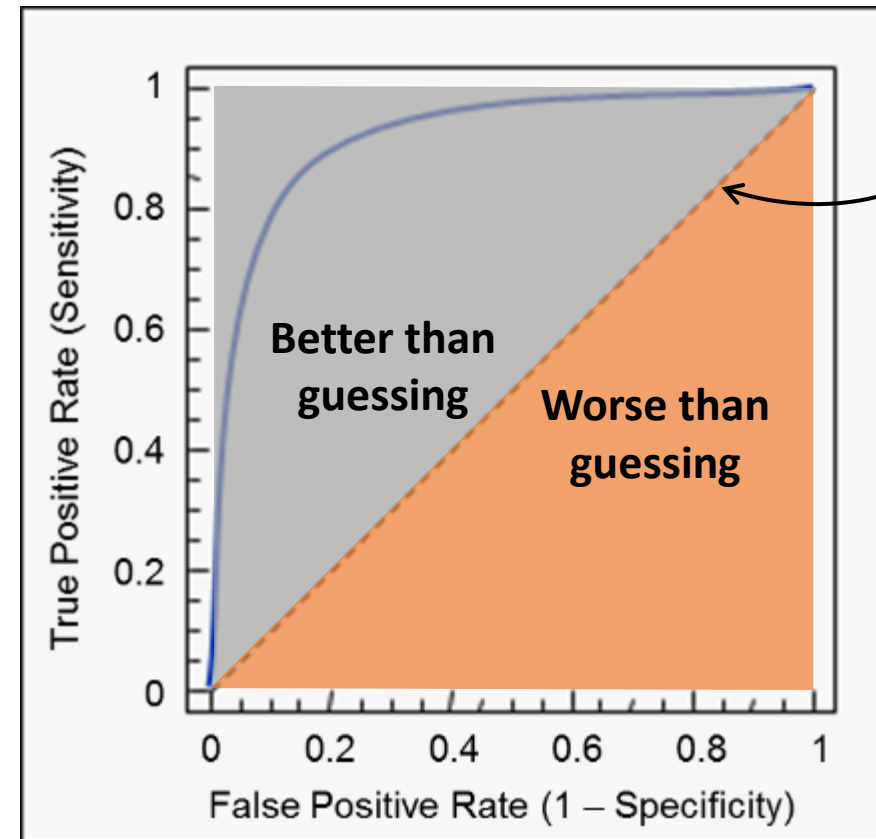
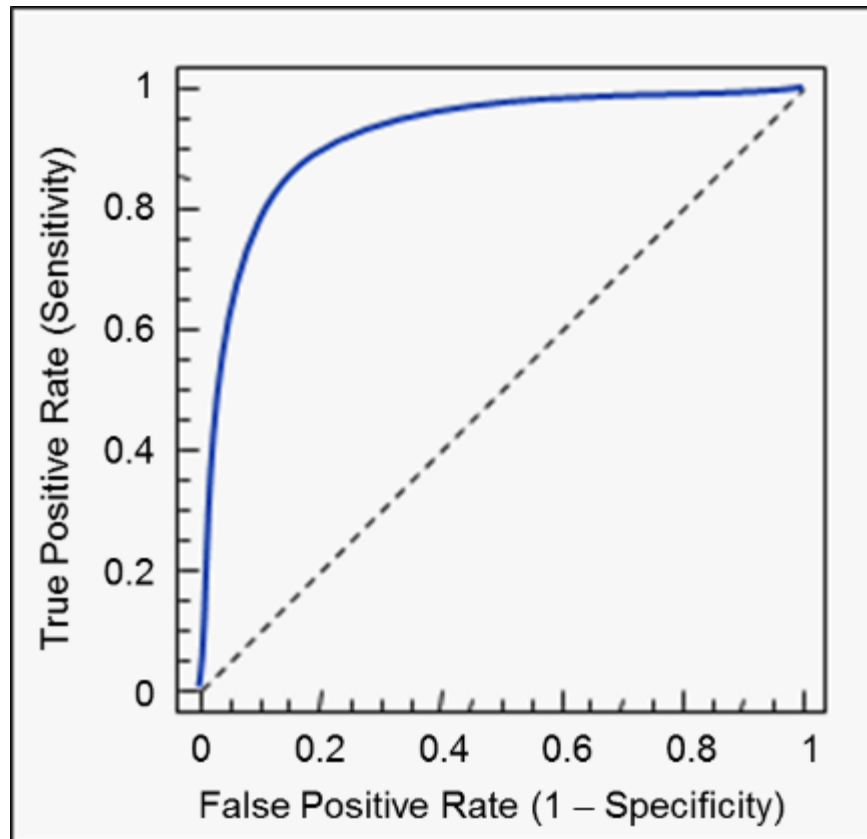
- **Receiver Operating Characteristic (ROC) curve**
illustrates the performance of a **binary** classifier as its discrimination threshold is **varied**.



For every possible cut-off point or criterion value you select to discriminate between the two groups

Model performance

- The **ROC** analysis provides tools to select possibly optimal models and to discard suboptimal ones independently from (and prior to specifying) the cost context or the class distribution.

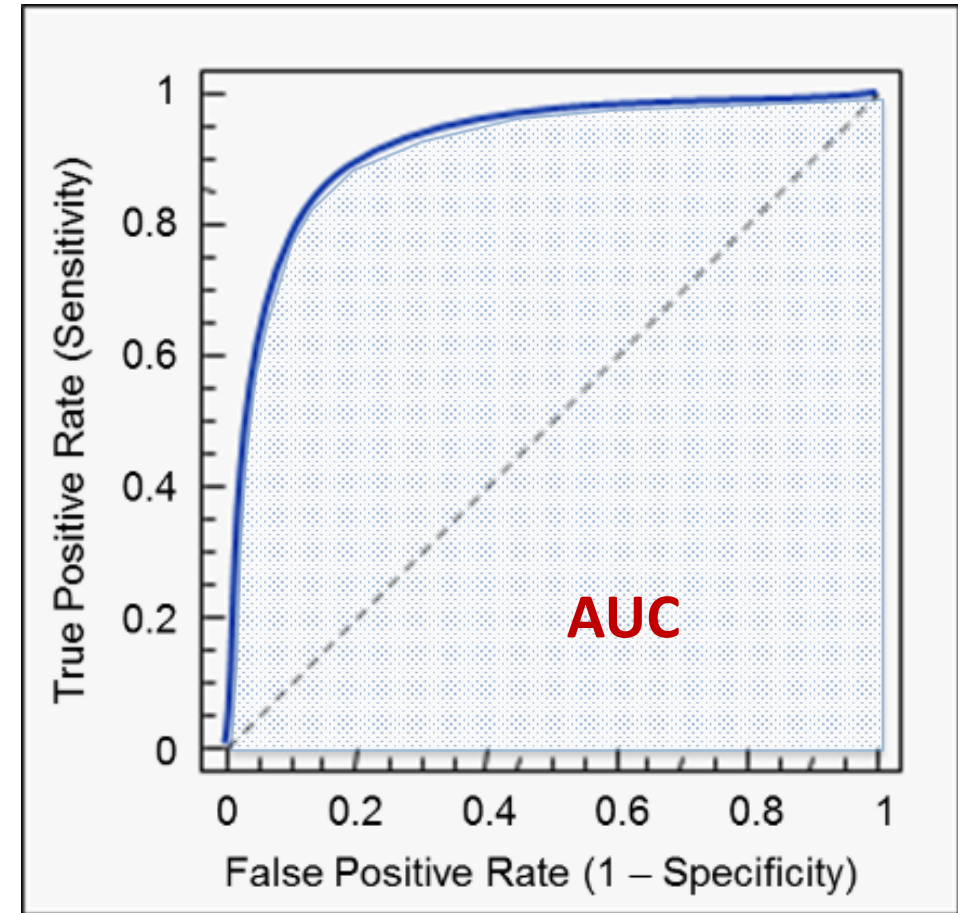


Model performance

- The Area Under the ROC curve (AUROC) or simply **AUC** measures the **probability** that a classifier will **rank** a randomly chosen positive instance **higher** than a randomly chosen negative one (assuming 'positive' ranks higher than 'negative')
- AUC **varies** between 0 and 1 — with an **uninformative** classifier yielding 0.5

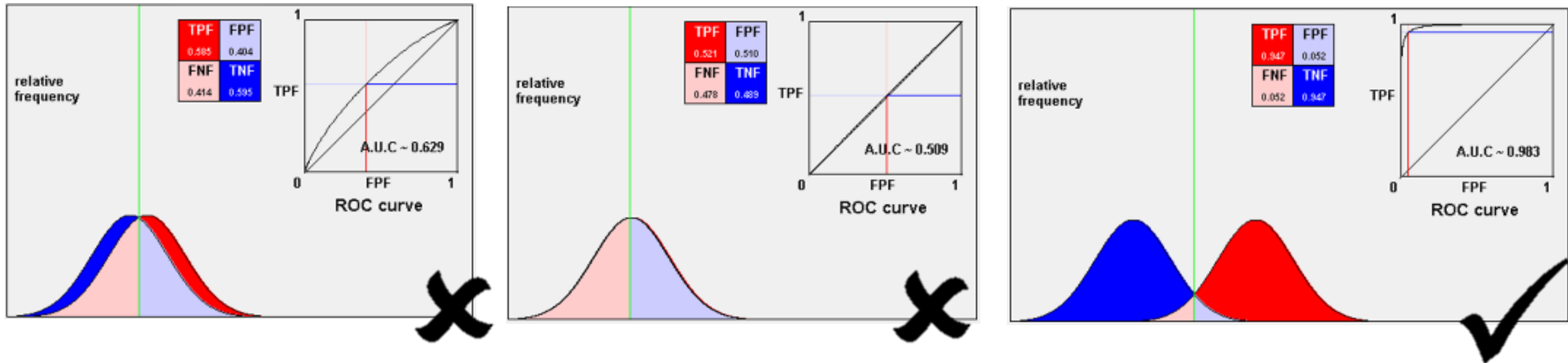
Area under the ROC curve: (0,1)

- 0.5 : uninformative classifier
- $\gg 0.5$: good classifier
- < 0.5 : useless



Model performance

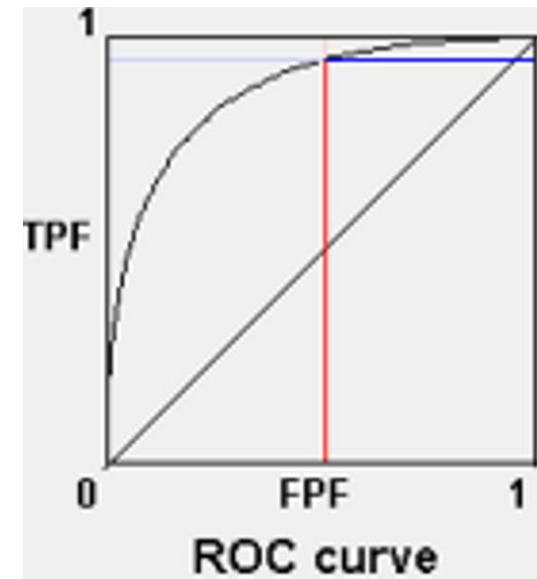
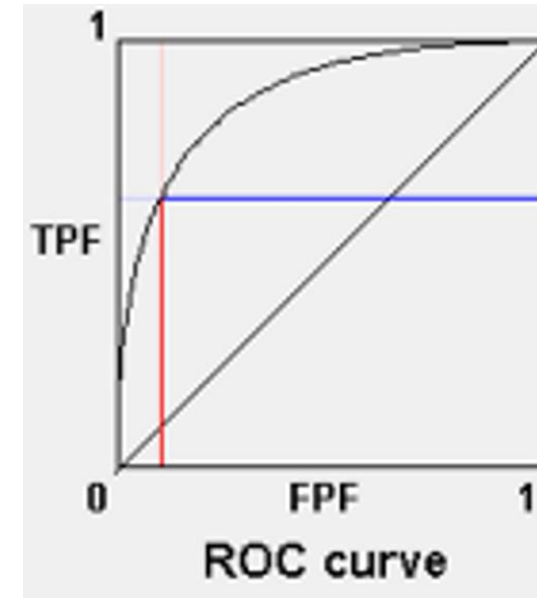
- ROC curve demonstrations: bad and good models



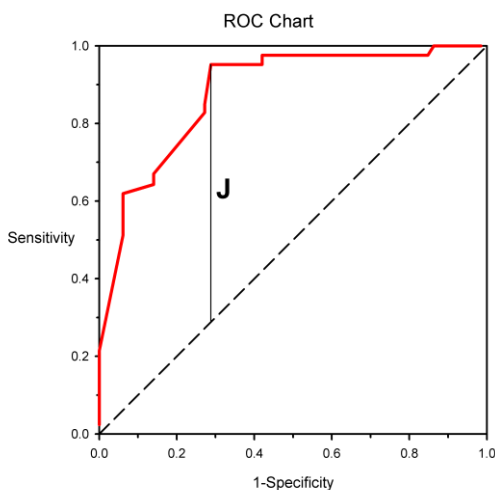
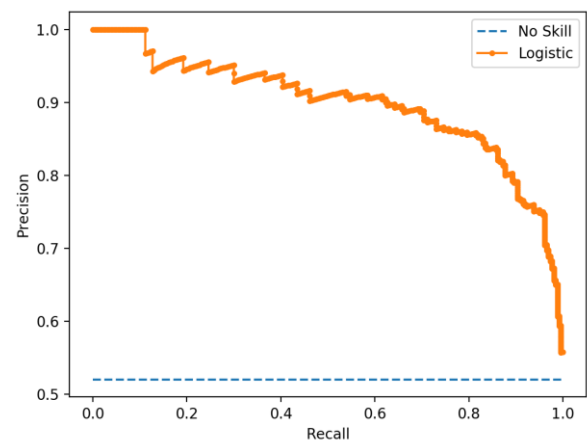
<https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc?hl=es-419>

Choosing the right cut-off point

- The cut-off point in a ROC curve is chosen based on the trade-off between the false positive rate and the true positive rate.
 - A **lower** false positive rate will result in **fewer** false alarms, but may also result in **more** missed detections.
 - A **higher** true positive rate will result in **fewer** missed detections, but may also result in **more** false alarms.
- The **optimal** cut-off point is the point on the ROC curve that **maximises** the true positive rate while **minimising** the false positive rate.
- It is also important to consider the **cost** of false positives and false negatives in the specific application when choosing a cut-off point.



Precision-recall (PR) curves



- It is a plot of the precision (y-axis) and the recall (x-axis) for different thresholds.
- Reviewing both precision and recall is useful in cases where there is an imbalance in the observations between the two classes. F-score (or F-measure) is a measure of accuracy on the positive class.

$$F_1 = 2 \frac{precision \times recall}{precision + recall} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

- The range of possible F-scores is $0 \leq F_1 \leq 1$. Where $F_1 = 1$ indicates perfect precision and recall.
- The Area Under the PR curve (AUC-PR) can also be estimated.
- Youden (J). Finds the threshold that **balances sensitivity and specificity** equally.

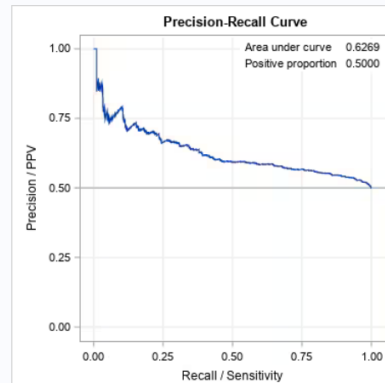
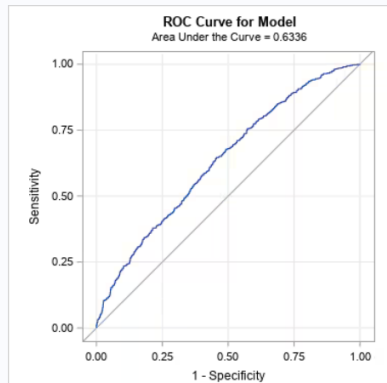
$$J = TPR - FPR$$

- Matthews correlation coefficient(MCC). +1 indicates perfect prediction, 0 random guessing, and -1 total disagreement between prediction and observation.

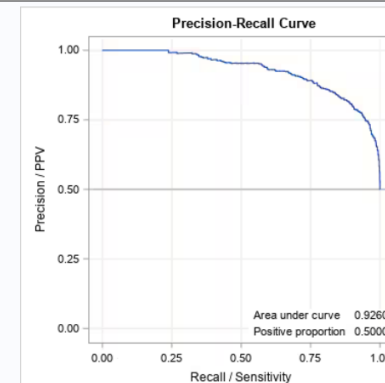
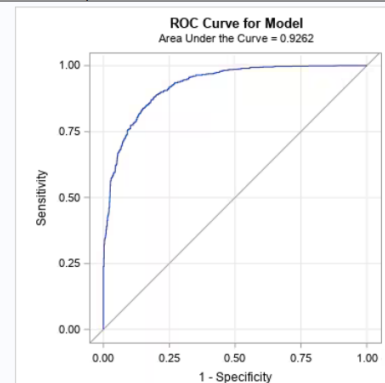
$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Metric	Best for data	What it balances
Youden's J	Balanced classes (50/50)	Sensitivity & Specificity equally
F1 Score	Imbalanced (minority class matters)	Precision & Recall (positive class only)
MCC	Any (especially extreme imbalance)	All four confusion matrix cells

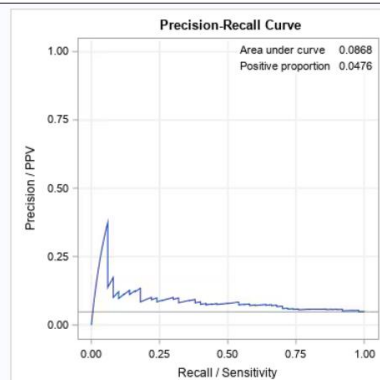
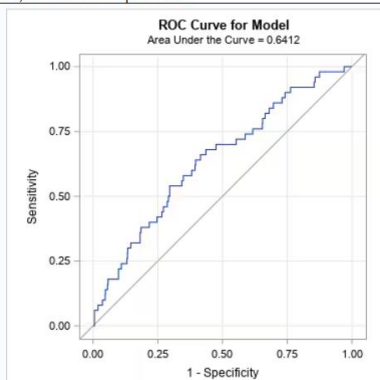
Balanced Data, Moderate Separation



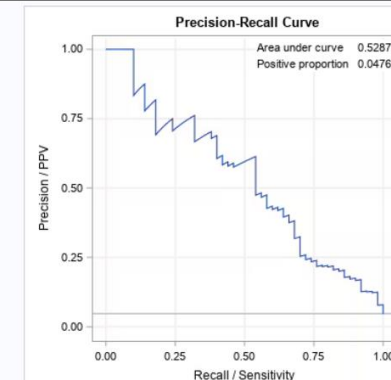
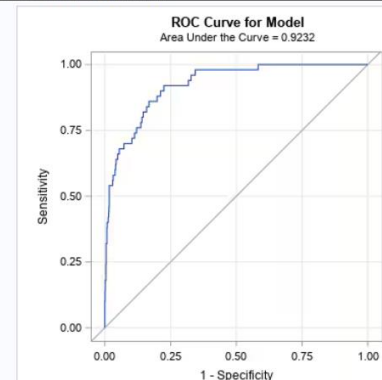
Balanced Data, Good Separation



Unbalanced Data, Moderate Separation



Unbalanced Data, Good Separation



```
from sklearn.metrics import roc_curve, precision_recall_curve, matthews_corrcoef
```

```
fpr, tpr, thresholds_roc = roc_curve(y_true, y_scores)
youden_idx = np.argmax(tpr - fpr)
best_youden = thresholds_roc[youden_idx]
```

```
mcc_scores = [matthews_corrcoef(y_true, y_scores >= th) for th in thresholds_roc]
best_mcc = thresholds_roc[np.argmax(mcc_scores)]
```

```
prec, rec, thresholds_pr = precision_recall_curve(y_true, y_scores)
f1 = 2 * (prec * rec) / (prec + rec + 1e-9)
best_f1 = thresholds_pr[np.argmax(f1)]
```

Summary

1. Classification and regression problems can be solved using ML models. We studied the simplest ML algorithms: linear regression and logistic regression.
2. A training set is used to build (train) the ML model. However, we need a different set (test set) to assess the generalisation abilities of the model. The test set is not seen by the model during the training process.
3. We studied several evaluation metrics for regression (e.g. RSS, RMSE, etc) and for classification (AUC, accuracy, etc)